



Digital Logic Design — Quick Revision Guide

Complete Summary of Topics, Processes, Laws & Equations

Course Code: CSE 211 | All-in-One Last-Minute Reference



Topic Index (15 Topics)

- | | | | |
|---|-----------------------------------|----|-----------------------------------|
| 1 | Number Systems & Base Conversions | 9 | Decoder & Encoder |
| 2 | Complements (r's & (r-1)'s) | 10 | Multiplexer (MUX) & Demultiplexer |
| 3 | Binary Arithmetic & Subtraction | 11 | Magnitude Comparator |
| 4 | Boolean Algebra Laws & Theorems | 12 | Priority Encoder |
| 5 | Logic Gates & Universal Gates | 13 | BCD, Excess-3 & Decimal Codes |
| 6 | SOP, POS, Minterm & Maxterm | 14 | Flip-Flops (Sequential Logic) |
| 7 | K-Map Simplification | 15 | Universality of NAND/NOR |
| 8 | Adders & Subtractors | ★ | Quick Formula Sheet |



Topic 1: Number Systems & Base Conversions

Number Systems Overview

System	Base	Digits Used	Example
Binary	2	0, 1	$(1011)_2$
Octal	8	0–7	$(157)_8$
Decimal	10	0–9	$(395)_{10}$
Hexadecimal	16	0–9, A(10)–F(15)	$(3FA)_{16}$



Process 1: Decimal → Any Base (r)

- Integer part:** Divide by base r repeatedly, collect remainders (read bottom-up).
- Fractional part:** Multiply by base r repeatedly, collect integer parts (read top-down).
- Combine both parts with the radix point.



Process 2: Any Base → Decimal

Multiply each digit by its **positional weight** (base raised to power of position).

$$N_{10} = \sum (\text{digit}_i \times \text{base}^i)$$

For $(1011.1)_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} = 11.5$



Process 3: Binary ↔ Octal/Hex (Direct Method)

Conversion	Group Size	Direction
Binary → Octal	3 bits	From radix point outward
Binary → Hex	4 bits	From radix point outward
Octal → Binary	Each digit → 3 bits	—
Hex → Binary	Each digit → 4 bits	—

Hex ↔ Binary Quick Reference

Hex	Binary	Hex	Binary
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100

5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

 **Tip:** Largest n-bit unsigned number = $2^n - 1$. For 16 bits = 65535 = FFFF.

Topic 2: Complements (r's & (r-1)'s)

Definition

For base r:

- **(r-1)'s complement** = subtract each digit from (r-1)
- **r's complement** = (r-1)'s complement + 1

System (r)	(r-1)'s Complement	r's Complement
Binary (r=2)	1's complement (flip all bits)	2's = 1's + 1
Octal (r=8)	7's complement (777 - N)	8's = 7's + 1
Decimal (r=10)	9's complement (999 - N)	10's = 9's + 1
Hex (r=16)	15's complement (FFF - N)	16's = 15's + 1

Process: Find 2's Complement of Binary Number

- 1 Find 1's complement (flip every bit: 0 → 1, 1 → 0).
- 2 Add 1 to the LSB.

Shortcut: Starting from LSB, copy bits until first 1; flip all bits after that.

Example: 2's complement of 1011001

1's = 0100110 → +1 → **0100111**

Quick Formulas:

1's comp of N = $(2^n - 1) - N$

2's comp of N = $2^n - N$

9's comp of N = $(10^n - 1) - N$

15's comp of N = $(16^n - 1) - N$

Topic 3: Binary Arithmetic & Subtraction

Binary Addition Rules

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Binary Subtraction Rules

A	B	Diff	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Process: Subtraction Using r's Complement ($M - N$)

- 1 Make M and N the **same number of digits** (pad zeros if needed).
- 2 Find **r's complement** of N (subtrahend).
- 3 **Add** $M + (\text{r's complement of N})$.
- 4 Check for end carry:
 - **5 End carry exists** → result is **positive**; **discard the carry**. Result is the answer.
 - **6 No end carry** → result is **negative**; take r's complement of result and add minus sign.

 **Memory Trick:** "End carry → keep & discard; No carry → flip & negate"

Topic 4: Boolean Algebra Laws & Theorems

Complete Boolean Laws Reference Table

Law / Postulate	OR Form (+)	AND Form (·)
Identity	$A + 0 = A$	$A \cdot 1 = A$
Null (Domination)	$A + 1 = 1$	$A \cdot 0 = 0$
Idempotent	$A + A = A$	$A \cdot A = A$
Complement	$A + A' = 1$	$A \cdot A' = 0$
Involution	$(A')' = A$	

Commutative	$A + B = B + A$	$A \cdot B = B \cdot A$
Associative	$A + (B + C) = (A + B) + C$	$A(BC) = (AB)C$
Distributive	$A(B + C) = AB + AC$	$A + BC = (A+B)(A+C)$
Absorption	$A + AB = A$	$A(A + B) = A$
De Morgan's	$(A + B)' = A' \cdot B'$	$(A \cdot B)' = A' + B'$
Consensus	$A + A'B = A + B$	$A(A' + B) = AB$
Redundancy	$AB + A'C + BC = AB + A'C$	

Process: Boolean Simplification Strategy

- 1 Look for **complements**: $A \cdot A' = 0$, $A + A' = 1$.
- 2 Look for **nulls/identities**: $A + 1 = 1$, $A \cdot 0 = 0$, $A \cdot 1 = A$.
- 3 Apply **De Morgan's** to break/combine bars over groups.
- 4 Apply **distributive** to factor or expand.
- 5 Apply **absorption**: $A + AB = A$.
- 6 Apply **consensus**: $A + A'B = A + B$.
- 7 Re-check at each step for further simplification opportunities.

Duality Principle

The **dual** of a Boolean expression is obtained by:

- Swapping AND $\leftrightarrow$ OR ($\cdot \leftrightarrow +$)
- Swapping 0 $\leftrightarrow$ 1
- Variables remain unchanged

Example: Dual of $(A + 0) \cdot (B \cdot 1 + C) = (A \cdot 1) + (B + 0) \cdot C$

Topic 5: Logic Gates — Symbols, Equations & Truth Tables

Gate	Equation	Truth Table (2-input, A·B)
NOT	$Y = A'$	$0 \rightarrow 1, 1 \rightarrow 0$
AND	$Y = A \cdot B$	$00 \rightarrow 0, 01 \rightarrow 0, 10 \rightarrow 0, 11 \rightarrow 1$
OR	$Y = A + B$	$00 \rightarrow 0, 01 \rightarrow 1, 10 \rightarrow 1, 11 \rightarrow 1$
NAND	$Y = (A \cdot B)'$	$00 \rightarrow 1, 01 \rightarrow 1, 10 \rightarrow 1, 11 \rightarrow 0$
NOR	$Y = (A + B)'$	$00 \rightarrow 1, 01 \rightarrow 0, 10 \rightarrow 0, 11 \rightarrow 0$
XOR	$Y = A \oplus B = A'B + AB'$	$00 \rightarrow 0, 01 \rightarrow 1, 10 \rightarrow 1, 11 \rightarrow 0$
XNOR	$Y = A \odot B = AB + A'B'$	$00 \rightarrow 1, 01 \rightarrow 0, 10 \rightarrow 0, 11 \rightarrow 1$

XOR/XNOR Important Properties

$$\begin{aligned}
 A \oplus 0 &= A & A \oplus 1 &= A' \\
 A \oplus A &= 0 & A \oplus A' &= 1 \\
 A \oplus B \oplus A &= B \text{ (Self-cancellation)} \\
 (A \oplus B)' &= A \odot B = A'B' + AB
 \end{aligned}$$

Topic 6: SOP, POS, Minterm & Maxterm

Key Definitions

- **Literal:** A variable or its complement (A, A', B, B'). The expression A'B + ABC has 5 literals.
- **Minterm (m):** A product term with all variables present (e.g., A'BC for 3 vars). Equals 1 for exactly one combination.
- **Maxterm (M):** A sum term with all variables present (e.g., A+B'+C). Equals 0 for exactly one combination.
- **SOP (Sum of Products):** OR of AND terms. Example: $F = AB + A'C$
- **POS (Product of Sums):** AND of OR terms. Example: $F = (A+B)(A'+C)$

Minterm & Maxterm Reference (3 variables)

m/M	A	B	C	Minterm (m)	Maxterm (M)
0	0	0	0	A'B'C'	A+B+C
1	0	0	1	A'B'C	A+B+C'
2	0	1	0	A'BC'	A+B'+C
3	0	1	1	A'BC	A+B'+C'
4	1	0	0	AB'C'	A'+B+C
5	1	0	1	AB'C	A'+B+C'

6	1	1	0	ABC'	A'+B'+C
7	1	1	1	ABC	A'+B'+C'



Process: SOP ↔ POS Conversion

- 1 Build the truth table for the function.
- 2 Identify minterms (rows where F=1) for SOP form: $F = \sum m(\dots)$.
- 3 Identify maxterms (rows where F=0) for POS form: $F = \prod M(\dots)$.
- 4 The minterm numbers and maxterm numbers are **complementary** — every row index is in exactly one.

Key Relationship:

For n variables (2^n total minterms): $\sum m(\text{used}) \leftrightarrow \prod M(\text{unused indices})$

Example: $F(A,B,C) = \sum m(1,3,5,7) \rightarrow F = \prod M(0,2,4,6)$



Process: Convert to Canonical (Standard) Form

SOP → Canonical SOP: Multiply each missing variable as $(X + X') = 1$.

Example: $AB \rightarrow AB \cdot (C+C') = ABC + ABC'$

POS → Canonical POS: Add $(X \cdot X') = 0$ to each term using distributive.

Example: $(A+B) \rightarrow (A+B+CC') = (A+B+C)(A+B+C')$

Topic 7: K-Map (Karnaugh Map) Simplification

K-Map Grouping Rules

1. Groups must contain **powers of 2** cells: 1, 2, 4, 8, 16.
2. Groups must be **rectangular** (horizontal/vertical) — no diagonals.
3. Each group must be **as large as possible**.
4. Use **fewest groups** that cover all 1's.
5. **Wrap-around** is allowed (sides connect; corners can group).
6. Cells can belong to **multiple groups**.
7. Use **don't cares** (×) only if they help form bigger groups.

K-Map Layouts (Gray Code Order)

2-Variable:

A\B	0	1
0	m0	m1
1	m2	m3

3-Variable:

A\BC	00	01	11	10
0	m0	m1	m3	m2
1	m4	m5	m7	m6

4-Variable:

AB\CD	00	01	11	10
00	m0	m1	m3	m2
01	m4	m5	m7	m6
11	m12	m13	m15	m14
10	m8	m9	m11	m10

Process: K-Map Simplification (SOP)

- 1 Draw the K-map structure (Gray code order!).
- 2 Place 1's at minterm positions, 0's at maxterm positions, × at don't-cares.
- 3 Find the **largest possible groups** covering all 1's (octets > quads > pairs > singles).
- 4 For each group, find the **common variables** (variables that don't change).
- 5 Write each group as a product term using common variables.
- 6 Combine all terms with OR to get the final SOP expression.

Group Size → Literals Eliminated:

Cells in group	Variables eliminated	Literals in term (n vars)
1	0	n
2	1	n - 1

4	2	$n - 2$
8	3	$n - 3$
16	4	$n - 4$

 **Tip:** For POS simplification, group 0's instead, and use **complemented variables that change to 0 in the group**.

Topic 8: Adders & Subtractors

Half Adder

Inputs: A, B | **Outputs:** Sum (S), Carry (C)

$$S = A \oplus B$$

$$C = A \cdot B$$

Built with: 1 XOR + 1 AND

Full Adder

Inputs: A, B, C_{in} | **Outputs:** Sum (S), Carry-out (C_{out})

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + (A \oplus B) \cdot C_{in} = AB + BC_{in} + AC_{in}$$

Built with: 2 Half Adders + 1 OR gate

Half Subtractor

Inputs: A, B | **Outputs:** Difference (D), Borrow (B_{out})

$$D = A \oplus B$$

$$B_{out} = A' \cdot B$$

Full Subtractor

Inputs: A, B, B_{in} | **Outputs:** Difference (D), Borrow-out (B_{out})

$$D = A \oplus B \oplus B_{in}$$

$$B_{out} = A'B + (A \oplus B)' \cdot B_{in} = A'B + A'B_{in} + BB_{in}$$

Built with: 2 Half Subtractors + 1 OR gate

 **Note:** Sum and Difference equations are **identical!** Only the carry/borrow logic differs.

4-bit Parallel Adder

Cascade of **4 Full Adders**: each FA's C_{out} feeds the next FA's C_{in} . Initial $C_0 = 0$.

Topic 9: Decoder & Encoder

Decoder

A **decoder** converts an n-bit binary input into 2^n output lines (only 1 active).
Each output corresponds to one minterm of the input.

2-to-4 Decoder (Active High)

A	B	D ₀	D ₁	D ₂	D ₃
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

$$D_0 = A'B' \quad D_1 = A'B \quad D_2 = AB' \quad D_3 = AB$$

Process: Implement Function with Decoder

- 1 Find SOP form (list minterms): $F = \Sigma m(\dots)$.
- 2 Use 3-to-8 decoder (or appropriate size).
- 3 OR together the decoder output lines corresponding to minterms.

Encoder

An **encoder** is the reverse of a decoder: 2^n inputs $\rightarrow$ n outputs (binary code).
Only one input should be active at a time (otherwise use a priority encoder).

Topic 10: Multiplexer (MUX) & Demultiplexer

A **MUX** selects 1 of 2^n data inputs based on n select lines, routing it to a single output.
A **DEMUX** does the reverse: 1 input $\rightarrow$ 1 of 2^n outputs.

MUX Output Equations

2×1 MUX:

$$Y = S' \cdot I_0 + S \cdot I_1$$

4×1 MUX:

$$Y = S_1'S_0'I_0 + S_1'S_0I_1 + S_1S_0'I_2 + S_1S_0I_3$$

8×1 MUX:

$$Y = \sum (S_2 S_1 S_0 \text{ minterms} \cdot I_i)$$

Building Larger MUX from Smaller

Goal	Construction	Total MUXes
4×1 from 2×1	Tree of 2 levels	3
8×1 from 2×1	Tree of 3 levels	7
16×1 from 4×1	Tree of 2 levels	5
16×1 from 2×1	Tree of 4 levels	15



Process: Implement Function Using MUX

- 1 For an n-variable function, use a $2^{n-1} \times 1$ MUX.
- 2 Connect (n-1) variables to select lines.
- 3 For each combination of selects, determine if F equals 0, 1, the last variable D, or D'.
- 4 Connect that value to the corresponding I_i input.



Topic 11: Magnitude Comparator

A **magnitude comparator** compares two binary numbers and produces 3 outputs: $A > B$, $A = B$, $A < B$.

1-bit Comparator

$$A > B: AB' \quad A = B: A \odot B = A'B' + AB \quad A < B: A'B$$

2-bit Comparator ($A = A_1A_0$, $B = B_1B_0$)

$$(A > B) = A_1B_1' + (A_1 \odot B_1) \cdot A_0B_0'$$

$$(A = B) = (A_1 \odot B_1) \cdot (A_0 \odot B_0)$$

$$(A < B) = A_1'B_1 + (A_1 \odot B_1) \cdot A_0'B_0$$

Topic 12: Priority Encoder

A **priority encoder** assigns priority to inputs. When multiple inputs are active, only the highest-priority input is encoded. Often includes a **Valid (V)** output.

4-input Priority Encoder (D_3 = highest priority by convention)

D_3	D_2	D_1	D_0	x	y	V
0	0	0	0	×	×	0
0	0	0	1	0	0	1
0	0	1	×	0	1	1
0	1	×	×	1	0	1
1	×	×	×	1	1	1

$$x = D_2 + D_3$$

$$y = D_3 + D_1 \cdot D_2'$$

$$V = D_0 + D_1 + D_2 + D_3$$

Process: Design Custom-Priority Encoder

- 1 List inputs in priority order (highest first).
- 2 Assign output codes (00, 01, 10, 11) by priority.
- 3 Build truth table — lower-priority inputs become don't-cares when higher-priority is active.
- 4 Derive equations for each output bit using K-map or direct logic.
- 5 V = OR of all inputs.

Topic 13: Decimal Codes (BCD, Excess-3)

Code Definitions

- **BCD (8421):** Each decimal digit (0-9) → 4-bit binary. Codes 1010-1111 are invalid.
- **Excess-3:** Add 3 to decimal digit, then convert to 4-bit binary. (e.g., $5+3=8 \rightarrow 1000$)
- **Gray Code:** Adjacent codes differ by only 1 bit (used in K-maps).
- **Parity bit:** Extra bit to make total 1's even (even parity) or odd (odd parity) for error detection.

Code Conversion Reference Table

Decimal	BCD (8421)	Excess-3	Gray Code
0	0000	0011	0000
1	0001	0100	0001

2	0010	0101	0011
3	0011	0110	0010
4	0100	0111	0110
5	0101	1000	0111
6	0110	1001	0101
7	0111	1010	0100
8	1000	1011	1100
9	1001	1100	1101

Process: Binary ↔ Gray Conversion

Binary to Gray:

$G_n = B_n$ (MSB stays the same)

$G_i = B_{i+1} \oplus B_i$ for other bits

Gray to Binary:

$B_n = G_n$

$B_i = B_{i+1} \oplus G_i$

Parity Bit Generation

Even parity bit: $P = D_1 \oplus D_2 \oplus \dots \oplus D_n$

Odd parity bit: $P = (D_1 \oplus D_2 \oplus \dots \oplus D_n)'$

Topic 14: Flip-Flops (Sequential Logic)

A **flip-flop** is a 1-bit memory element. Output depends on both current inputs and previous state. Building blocks of registers and counters.

Complete Flip-Flop Reference

Type	Inputs	Characteristic Equation	Main Use
SR	S, R	$Q(t+1) = S + R' \cdot Q(t)$	Basic memory
JK	J, K	$Q(t+1) = J \cdot Q'(t) + K' \cdot Q(t)$	Counters, general purpose
D	D	$Q(t+1) = D$	Data storage, registers
T	T	$Q(t+1) = T \oplus Q(t)$	Toggle, frequency divider

SR Flip-Flop Truth Table

S	R	$Q(t+1)$	State
0	0	$Q(t)$	No change
0	1	0	Reset
1	0	1	Set
1	1	?	Invalid

JK Flip-Flop Truth Table

J	K	$Q(t+1)$	State
0	0	$Q(t)$	No change
0	1	0	Reset
1	0	1	Set
1	1	$Q'(t)$	Toggle

D & T Flip-Flop Truth Tables

D Flip-Flop:

D	$Q(t+1)$
0	0
1	1

T Flip-Flop:

T	$Q(t+1)$
0	$Q(t)$
1	$Q'(t)$

Excitation Tables (For Sequential Design)

$Q(t) \rightarrow Q(t+1)$	SR	JK	D	T
$0 \rightarrow 0$	$0 \times$	$0 \times$	0	0

$0 \rightarrow 1$	10	$1 \times$	1	1
$1 \rightarrow 0$	01	$\times 1$	0	1
$1 \rightarrow 1$	$\times 0$	$\times 0$	1	0

Topic 15: Universality of NAND & NOR

A gate is **universal** if any Boolean function can be implemented using only that gate. Both **NAND** and **NOR** are universal.

NAND as Universal Gate

Gate	Implementation using NAND
NOT	$A \text{ NAND } A = A'$
AND	NAND followed by NAND-as-NOT: $[(A \cdot B)']' = AB$
OR	De Morgan: $A+B = (A' \cdot B')' = \text{NAND}(A', B')$

NOR as Universal Gate

Gate	Implementation using NOR
NOT	$A \text{ NOR } A = A'$
OR	NOR followed by NOR-as-NOT: $[(A+B)']' = A+B$
AND	De Morgan: $A \cdot B = (A'+B')' = \text{NOR}(A', B')$

★ Master Formula Sheet (Last-Minute Revision)

📐 Boolean Algebra Quick Reference

Identity: $A+0=A$, $A \cdot 1=A$ | Null: $A+1=1$, $A \cdot 0=0$
Idempotent: $A+A=A$, $A \cdot A=A$ | Complement: $A+A'=1$, $A \cdot A'=0$
Involution: $(A')'=A$ | Absorption: $A+AB=A$, $A(A+B)=A$
Consensus: $A+A'B=A+B$ | De Morgan: $(A+B)'=A'B'$, $(AB)'=A'+B'$
Distributive: $A(B+C)=AB+AC$, $A+BC=(A+B)(A+C)$

1234 Number System Quick Reference

Largest n-bit unsigned: 2^n-1
1's complement: Flip all bits
2's complement: 1's + 1
9's complement: $999...-N$ | 10's complement: 9's + 1
15's complement: $FFF...-N$ | 16's complement: 15's + 1

⚡ Combinational Circuits Quick Reference

Half Adder: $S=A \oplus B$, $C=AB$
Full Adder: $S=A \oplus B \oplus C_{in}$, $C_{out}=AB+BC_{in}+AC_{in}$
Half Subtractor: $D=A \oplus B$, $B_{out}=A'B$
Full Subtractor: $D=A \oplus B \oplus B_{in}$, $B_{out}=A'B+A'B_{in}+BB_{in}$
2x1 MUX: $Y=S'I_0+SI_1$
4x1 MUX: $Y=S_1'S_0'I_0+S_1'S_0I_1+S_1S_0'I_2+S_1S_0I_3$

🔄 Flip-Flop Quick Reference

SR: $Q(t+1) = S + R'Q$ | constraint: $SR=0$
JK: $Q(t+1) = JQ' + K'Q$
D: $Q(t+1) = D$
T: $Q(t+1) = T \oplus Q$

🎯 Special Functions

XOR: $A \oplus B = A'B + AB'$ | $A \oplus 0=A$, $A \oplus 1=A'$, $A \oplus A=0$
XNOR: $A \odot B = AB + A'B' = (A \oplus B)'$
NAND universality: $\text{NOT}(A \cdot A)$, $\text{AND}[(AB)']'$, $\text{OR}[\text{NAND}(A', B')]$
NOR universality: $\text{NOT}(A+A)$, $\text{OR}[(A+B)']'$, $\text{AND}[\text{NOR}(A', B')]$

Exam Strategy & Common Pitfalls

Standard Solution Format (Use this for every question):

1. **State the problem** and identify input/output variables.
2. **Build truth table** (2^n rows for n inputs).
3. **Identify minterms/maxterms** for SOP/POS.
4. **Draw K-map** in Gray code order.
5. **Find optimal groupings** (largest possible).
6. **Write simplified Boolean expression.**
7. **Draw logic diagram** using basic gates.
8. **Verify** with one or two truth table rows.

Common Mistakes to Avoid:

- Forgetting Gray code order in K-maps (00, 01, 11, 10 — NOT 00, 01, 10, 11).
- Not making groups **large enough** in K-maps.
- Forgetting to include carry-in/borrow-in for full versions.
- Mixing up SOP and POS minterm/maxterm indexing.
- In subtraction by complement: forgetting to **discard** end carry when result is positive.
- Forgetting NOT gates for complemented variables in logic diagrams.
- Not citing Boolean laws by name when simplifying.

Time Management (2-hour exam, 4 questions):

- ~30 minutes per question (10 marks).
- Spend 2-3 mins reading and planning each question first.
- If stuck on a part, move on and return later.
- Reserve last 10 minutes to review answers.

Presentation Tips:

- Show **every step** — partial credit for working.
- Label all logic gates clearly (AND, OR, NOT, NAND, etc.).
- Number truth table rows for easy minterm reference.
- Box your **final answer** for clarity.
- Cite Boolean law names (e.g., "by Distributive law", "by Absorption").